Streaming Communication for Scalable Data Exchange in DYNAMOS

Ruben Stoop
Complex Cyber-Infrastructure (CCI)
University of Amsterdam
Amsterdam, the Netherlands

***Abstract*—This paper investigates how streaming communication can be introduced into DYNAMOS, a policy-driven middleware for data exchange based on dynamically composed microservices. The current system relies on unary gRPC request-response interactions, which become increasingly inefficient for large or continuous transfers. The study combines a literature-guided comparison of candidate communication mechanisms with an initial controlled benchmark that establishes a gRPC reference baseline in a common *Producer* → *Transformer* → *Sink* pipeline. Phase 1 evaluates transfer efficiency, latency stability, overload behaviour, recovery characteristics, and resource cost. Preliminary results show that client-streaming gRPC provides substantially higher sustained throughput and steadier pacing for long-running transfers, while unary gRPC maintains lower per-message latency and cleaner restart behaviour. These findings establish a structured reference point for later comparison with broker-based alternatives such as Kafka, RabbitMQ streams, and NATS JetStream.**

***Index Terms*—streaming communication, microservices, gRPC, event streaming, policy-driven middleware, DYNAMOS**

I. Introduction

Many contemporary distributed systems operate in environments where data and computational resources are owned by different organizations. In such settings, data exchange must adhere to legal agreements, privacy regulations, and organizational policies. These constraints are common in domains such as healthcare, scientific research, and public-sector infrastructures, where unrestricted data exchange is not acceptable.

DYNAMOS (Dynamically Adaptive Microservice-based Operating System) is a data exchange middleware developed by the Complex Cyber-Infrastructure (CCI) group at the University of Amsterdam. The system implements a set of recurring data sharing archetypes that describe how data and computation are distributed among participants under policy constraints. DYNAMOS realizes these archetypes by dynamically composing chains of microservices that execute data sharing requests in compliance with formally evaluated policies. The middleware is designed to be adaptive, allowing architectural and execution decisions to change at runtime in response to policy requirements.

The current implementation of DYNAMOS uses gRPC for inter-service communication and relies exclusively on atomic request–response interactions. This communication model is suitable for small or bounded data transfers but becomes limiting when applied to large-scale or continuous data exchange. In practice, data in distributed systems is often produced and consumed incrementally, for example in sensor data streams, distributed analytics pipelines, or staged processing of large research datasets.

When request–response communication is used in such scenarios, systems must either buffer large volumes of data before transmission or split transfers into multiple independent requests. Buffering increases memory usage and can lead to idle time between producers and consumers. Repeated request–response interactions introduce additional coordination overhead. Both effects negatively impact scalability and resource utilization as data volumes and the number of participating services increase.

Streaming-based communication has been proposed as a solution to these limitations. Streaming allows data to be transferred and processed incrementally, enabling overlap between computation and communication and reducing peak memory usage. gRPC supports several streaming interaction patterns, including client-side, server-side, and bidirectional streaming.

Integrating streaming communication into DYNAMOS is not straightforward. Streaming relies on long-lived connections and continuous data flows, which must be reconciled with DYNAMOS' policy-driven execution model, dynamic microservice composition, orchestration decisions, and fault handling mechanisms. Without careful design, the introduction of streaming may interfere with policy enforcement or reduce the ability to evaluate and reason about system behavior.

The goal of this thesis is to investigate how streaming communication can be incorporated into DYNAMOS in a way that improves scalability and performance while preserving policy compliance and adaptive behavior. This involves analyzing existing streaming communication mechanisms, designing architectural extensions to the middleware, implementing streaming support, and evaluating the resulting system under realistic workloads.

The main research question addressed in this thesis is how streaming communication can be integrated into a dynamically adaptive, policy-driven microservice-based data exchange middleware such as DYNAMOS while preserving compliance guarantees and improving scalability and performance. To answer this overarching question, the thesis is structured around three sub-questions: which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-

based systems such as DYNAMOS; how streaming communication can be integrated into a microservice-based architecture while preserving dynamic service composition, policy-driven orchestration, and isolation guarantees; and what impact streaming communication has on scalability, performance, and robustness compared to a traditional request–response approach under realistic workloads in a microservice-based architecture.

II. Related Work

This section reviews existing literature related to streaming communication in microservice-based systems, with a particular focus on topics that are central to this thesis: streaming communication technologies and interaction models, architectural integration in adaptive microservice middleware, policy enforcement under continuous data exchange, resilience mechanisms for streaming workflows, and validation practices with respect to scalability and performance.

A. Streaming communication technologies and interaction models

DYNAMOS is implemented as a dynamically composed microservice system deployed on Kubernetes, using gRPC for inter-service communication and RabbitMQ for asynchronous messaging [1], [2]. This constrains the design space for introducing streaming functionality to a limited set of technologies that are compatible with containerized microservices and cloud-native deployment models. The primary question addressed in this body of work is which streaming communication mechanisms are suitable for large-scale or continuous data exchange, and under which conditions direct streaming RPC should be preferred over broker-based approaches.

gRPC provides native support for streaming through client-streaming, server-streaming, and bidirectional streaming RPCs. These interaction patterns differ in message flow control, lifetime of connections, buffering behaviour, and error propagation semantics. In the context of DYNAMOS, the choice of streaming pattern is not merely an API concern but has architectural consequences, as it affects memory usage, cancellation behaviour, and the ability to overlap communication and computation. The original DYNAMOS thesis identifies client-streaming as a promising mechanism for transferring large datasets incrementally, reducing peak memory pressure at the cost of increased handling complexity [1].

However, the use of long-lived streaming connections introduces operational challenges that are largely absent in unary request–response interactions. Starck and Ghofrani show that gRPC streaming complicates load balancing in Kubernetes environments, as persistent connections can cause traffic to become unevenly distributed across replicas [3]. This observation is particularly relevant for systems such as DYNAMOS that rely on horizontal scaling and ephemeral execution units. Complementary benchmarking work demonstrates that the performance characteristics of streaming gRPC are sensitive to runtime configuration and virtualization overhead, with measurable effects on latency and resource utilisation in containerized environments [4].

An alternative to direct service-to-service streaming is to externalise the data plane to a messaging or event streaming platform. Broker-based communication decouples producers and consumers in time, delegates buffering and persistence to the broker, and can simplify fan-out scenarios. Comparative benchmarks of Kafka, RabbitMQ, NATS, and ActiveMQ Artemis show that different brokers dominate under different workload characteristics, reinforcing that broker choice is highly workload dependent [5]. A cloud-native literature review comparing Kafka, Pulsar, and RabbitMQ further highlights trade-offs between throughput, reliability, operational complexity, and Kubernetes integration [6]. While these studies are not conducted in the context of policy-driven middleware, they provide structured criteria that are directly applicable when evaluating broker-based alternatives for DYNAMOS.

Several studies also emphasize that streaming design is influenced not only by technology choice but by the underlying communication model. DataXc contrasts push-based streaming approaches with pull-based designs and reports improved latency stability and throughput under consumer-controlled flow [7]. Although the application domain differs, the distinction between producer-driven and consumer-driven streaming is directly relevant to backpressure management and resource utilisation in DYNAMOS-compatible workflows.

Overall, existing literature provides a broad comparison of streaming technologies and interaction patterns, but these comparisons are typically conducted in generic microservice or stream-processing settings. There is limited work that evaluates these mechanisms under the specific constraints of dynamically composed, policy-driven middleware with ephemeral execution units, leaving open questions regarding their suitability in such environments.

B. Architectural integration of streaming in adaptive microservice middleware

Beyond the selection of a streaming mechanism, integrating streaming communication into an adaptive microservice architecture raises architectural questions related to modularity, orchestration, and lifecycle management. DYNAMOS is explicitly designed around dynamic microservice composition, where execution chains are generated and deployed per request based on policy evaluation and system state [1], [2]. These chains are short-lived by design, which improves isolation and compliance guarantees but complicates the management of long-lived communication channels.

Communication in DYNAMOS is abstracted through a sidecar-based architecture, decoupling communication concerns from core microservice logic. The sidecar pattern is widely used in cloud-native systems to address cross-cutting concerns such as observability, security, and traffic management. Figueira and Coutinho demonstrate how sidecars can be integrated into self-adaptive microservice architectures on Kubernetes, enabling runtime adaptation without violating

service modularity [8]. While their work does not explicitly address streaming data transfer, it illustrates how communication behaviour can be adapted dynamically at runtime.

Several middleware frameworks operationalise sidecars as first-class runtime components. Dapr provides a protocol-agnostic communication abstraction that supports service invocation, pub/sub, and state management through a sidecar runtime [8]. Dapr demonstrates that heterogeneous communication paradigms, including streaming and messaging, can be unified behind a stable interface. However, Dapr assumes relatively static service topologies and does not address dynamic chain composition driven by policy evaluation, limiting its applicability as a direct solution for DYNAMOS.

Existing architectural work therefore supports the feasibility of integrating streaming into sidecar-based microservice systems but does not address the combination of long-lived streaming connections with ephemeral, dynamically composed execution chains. This limitation is particularly relevant in systems that combine adaptivity with strict policy enforcement requirements.

C. Policy enforcement and compliance under continuous data exchange

Policy enforcement is a defining characteristic of DYNAMOS, where access rights, execution locations, and data-flow constraints are evaluated prior to execution and enforced through dynamically composed microservice chains [1], [2]. In the current architecture, policy enforcement is implicitly scoped to bounded request–response interactions. Introducing streaming communication challenges this assumption, as data transfer may span longer periods during which policies, system state, or execution context may change.

Neumaier et al. propose a policy-aware architecture for decentralized dataset exchange that emphasizes continuous monitoring and runtime validation against machine-readable policies [9]. While their work does not explicitly target streaming RPC, it introduces an important distinction between one-time policy clearance and ongoing compliance monitoring, which becomes critical when data is exchanged incrementally over long-lived connections.

At the communication layer, several studies propose enforcing security and access control through proxies or sidecars. Thiyagarajan et al. present a proxy-based approach for securing gRPC communication using mTLS, JWT authentication, and RBAC [10]. Although the focus is on security rather than data usage policy, the architectural approach is relevant, as it suggests that enforcement can be applied at communication boundaries and potentially at message granularity.

Despite these contributions, there is limited guidance on how policy enforcement should interact with long-lived streaming connections in systems that dynamically compose execution chains. In particular, existing work does not address how policy violations should be detected or handled mid-stream, or how streams should be terminated or adapted when compliance conditions change. These limitations highlight open challenges when introducing streaming into policy-driven middleware such as DYNAMOS.

D. Resilience, backpressure, and fault handling in streaming workflows

Streaming-based communication introduces operational challenges that are less pronounced in request–response systems. Long-lived data flows must tolerate partial failures, adapt to fluctuating load, and prevent uncontrolled resource growth through effective backpressure mechanisms. These challenges are compounded in DYNAMOS by dynamic microservice composition and ephemeral execution units.

Reactive microservice architectures emphasize asynchronous communication, non-blocking execution, and fault isolation as mechanisms for improving resilience under variable load [11]. Although this work does not focus on streaming RPC, its principles are directly applicable to streaming workflows, where failures and backpressure must be managed continuously rather than per request.

Studies on high-volume data processing in microservices highlight the importance of explicit flow control and decoupling between producers and consumers. Guntupalli and Vamshi show that event-driven architectures combined with messaging systems can mitigate bottlenecks and stabilize performance under sustained data rates [12]. While these systems typically rely on brokers rather than direct streaming RPC, they reinforce the need for explicit backpressure mechanisms in streaming data paths.

From a systems perspective, Gan and Delimitrou demonstrate that communication overhead and tail latency dominate the performance of large-scale microservice deployments [13]. Their findings underline that resilience and scalability issues often emerge from inter-service communication rather than computation, an effect that is amplified in streaming scenarios.

Existing literature provides well-established principles for resilience and fault tolerance in microservice systems, but these are typically studied either in request-oriented architectures or broker-based streaming platforms. There is limited work examining how these mechanisms interact with direct streaming RPC in dynamically composed microservice chains, leaving questions about how failures and backpressure should be handled in dynamically composed microservice systems.

E. Validation practices, scalability, and performance evaluation

Empirical evaluation of streaming communication often focuses on comparing streaming and non-streaming interaction styles in terms of throughput, latency, and resource usage. Several benchmarking studies show that streaming RPCs outperform unary calls for large payloads and continuous data flows by enabling incremental processing and reducing buffering overhead [4]. Similar conclusions are drawn in broader comparisons between REST and gRPC, where binary protocols combined with streaming semantics demonstrate superior performance under load [14], [15].

Scalability in containerized streaming systems has been studied through systematic mapping and empirical evaluations. A mapping study on performance analysis techniques for streaming workloads in containerized microservices identifies network contention, scheduling overhead, and coordination costs as primary scalability bottlenecks [16]. Empirical studies of streaming pipelines deployed on Kubernetes further show that streaming approaches scale more gracefully than request-response models when concurrency increases, provided that backpressure and flow control are properly configured [3].

However, most existing evaluations assume static microservice topologies and fixed execution paths. Prior performance studies rarely consider adaptive middleware that dynamically constructs execution chains based on policy evaluation. As a result, there is limited empirical insight into how streaming communication scales when embedded within dynamically adaptive, policy-driven systems such as DYNAMOS. This lack of combined evaluation of streaming, scalability, and adaptivity motivates the experimental focus of this thesis.

III. Phase 1: Survey and Initial Benchmarking of Candidate Streaming Mechanisms

Phase 1 addresses *RQ1*: which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS? The phase combines a literature-backed comparison of candidate mechanisms with a first controlled benchmark that establishes a baseline within the gRPC family before broker-based alternatives are added.

A. gRPC streaming

According to the official gRPC documentation, gRPC is based on service definitions in Protocol Buffers and supports four RPC styles: unary, server-streaming, client-streaming, and bidirectional streaming. For streaming calls, message ordering is preserved within each individual RPC, while deadlines, cancellation, metadata, and both synchronous and asynchronous APIs are supported by the runtime [17]. These properties make gRPC streaming the most direct extension of the current DYNAMOS communication model.

For microservice chains with predominantly point-to-point data transfer, client-streaming is the most natural first candidate because it lets a sender incrementally push chunks or records to a single downstream consumer. Bidirectional streaming becomes relevant when the receiver should provide progress feedback, pacing information, or explicit acknowledgements during transfer. The main advantages of gRPC streaming are low protocol overhead, direct reuse of protobuf contracts, and the absence of an additional broker layer. The disadvantages are that sender and receiver remain temporally coupled, durable replay is not provided by default, and backlog handling or crash recovery must be implemented at application level. Even so, prior work on asynchronous gRPC streaming demonstrates that carefully designed protocols can support efficient and even exactly-once style delivery patterns, which makes gRPC a strong experimental baseline rather than merely the status-quo option [4], [14], [18].

B. Apache Kafka

Apache Kafka is officially described as an open-source distributed event-streaming platform for high-performance pipelines, streaming analytics, data integration, and mission-critical applications. Its published core capabilities emphasise high throughput, durable storage, high availability, built-in stream processing, and large-scale horizontal scalability [19]. In architectural terms, Kafka exposes an append-only log abstraction in which messages are written to topics and partitions, allowing replay and independent consumption by multiple downstream readers [20].

Kafka is therefore the strongest candidate when temporal decoupling, replayability, or large consumer backlogs matter more than direct point-to-point simplicity. Its advantages are durable retention, repeated reads by independent consumers, and a mature ecosystem for analytics and downstream integration. The disadvantages are the extra operational footprint of a broker cluster, partition-local rather than global ordering, and a non-trivial tuning burden around acknowledgements, partitioning, batching, and replication. Recent review work also notes that Kafka benchmarks are frequently hard to reproduce because configuration details are underreported, which strengthens the case for a carefully controlled thesis benchmark [20]–[22].

C. RabbitMQ

Earlier comparative work positions RabbitMQ as a routing-oriented messaging system that offers flexible exchanges, acknowledgements, and strong support for enterprise-style asynchronous messaging [20], [23]. Current RabbitMQ documentation additionally introduces *streams* as an append-only, non-destructive, always persistent and replicated data structure that complements traditional queues rather than replacing them. Stream consumers can attach at arbitrary offsets or timestamps, replay data, and scale out via super streams when partitioning is needed [24].

This makes RabbitMQ attractive as a hybrid comparison point between classic broker semantics and newer log-like stream semantics within the same ecosystem. Its advantages are flexible routing, explicit acknowledgements, and a gradual migration path from queue-oriented messaging to append-only streams. The limitations are that classic RabbitMQ semantics are not optimised for replay-heavy logs, while stream mode introduces its own replication and partitioning trade-offs. RabbitMQ should therefore be evaluated not only on raw throughput, but also on how well it balances routing flexibility, operational continuity, and stream-specific behaviour [20], [23], [24].

D. NATS with JetStream persistence

The NATS ecosystem deserves separate consideration because it targets lightweight, location-transparent distributed communication. Official NATS documentation presents core

NATS as a connective technology for modern distributed systems with publish/subscribe and request/reply communication, while JetStream adds persistence, replay, replication, and consumer state on top of the base message bus [25]–[27]. JetStream explicitly supports replay from the beginning of a stream, from a specific sequence number, or from a time-based position, and it offers both push-based and pull-based consumers for different workload profiles [27].

For the thesis, NATS is interesting because it represents a lightweight alternative to Kafka and RabbitMQ rather than a direct substitute for either. JetStream supports durable streams, replay, replication, and scalable pull consumers, while core NATS preserves a simple request/reply and pub/sub model that is attractive for microservice systems [26], [27]. A key complication is that the literature still often discusses the older NATS Streaming layer rather than JetStream, which means published results do not always map perfectly to the current product. That mismatch should be made explicit in the thesis: the experiment should evaluate modern JetStream, but the literature review must interpret older NATS Streaming results with caution [22], [23]. NATS with JetStream is most promising when lightweight deployment, elastic scaling, and a smaller operational surface are valued more highly than Kafka's larger ecosystem and benchmarking history.

E. Phase 1 Experimental Design

1) Methodological positioning: Phase 1 is not presented as a standard benchmark protocol copied directly from one prior study, nor as a completely novel framework. Instead, it adapts recurring benchmarking principles from streaming-system evaluation, distributed stream processing studies, and comparative IPC experiments to the specific constraints of DYNAMOS. In particular, the controlled multi-stage pipeline follows the style of prior streaming-pipeline evaluations such as DataXc [7]; the insistence on identical business logic across mechanisms follows comparative benchmarking practice in distributed systems [28], [29]; and the emphasis on throughput, latency, concurrency, resource pressure, and recovery reflects recurring evaluation dimensions in the literature [16], [22]. The resulting design is therefore best understood as a literature-guided, thesis-specific operationalization of established evaluation concerns.

2) Evaluation scope: The purpose of Phase 1 is not to identify a universally best communication mechanism in the abstract. Instead, it narrows the design space for DYNAMOS by asking which communication model best fits which workload condition and which operational constraint. The central question is therefore: *which streaming mechanism best supports large, ordered, and concurrent inter-service data transfer in a microservice pipeline under varying workload intensity, resource pressure, and recovery requirements?*

This framing matters for interpretation. The chapter body focuses on the argument and on the most informative comparisons, while exhaustive preset definitions, full run matrices, and supplementary plots are better treated as appendix material once the multi-technology comparison is complete.

3) Common testbed and fairness constraints: To ensure that observed differences can be attributed to the communication mechanism rather than to application logic, all candidate mechanisms should be evaluated with the same payload schema, the same transformation logic, and the same deployment environment. The common testbed is a Go-based microservice chain deployed on Kubernetes with three services: *Producer* $\rightarrow$ *Transformer* $\rightarrow$ *Sink*. This topology is small enough to reason about directly, but still rich enough to expose backpressure, buffering, and multi-hop propagation effects [7], [28].

The mechanism-specific mappings should remain as close as possible to one another. For gRPC, the default implementation should use client-streaming, with bidirectional streaming reserved for an extended scenario. For Kafka, messages should be keyed by flow identifier so that per-flow ordering is preserved within a partition. For RabbitMQ, the preferred setup is a stream rather than a classic queue, with super streams used only if scale-out beyond a single stream is necessary. For NATS, JetStream should be used with explicit acknowledgements and pull consumers. Holding business logic and payload shape constant is essential, because otherwise performance differences could be caused by application behaviour instead of the communication layer itself [28], [29].

The service logic is intentionally simple. The producer emits synthetic records or chunked data, the transformer applies a deterministic lightweight operation such as validation, filtering, or hashing, and the sink records arrival times, latency summaries, and correctness counters. This keeps the benchmark focused on communication behaviour rather than on domain-specific business processing.

4) Evaluation dimensions: The literature supports evaluating streaming mechanisms along a small set of recurring dimensions rather than through a single headline metric. For Phase 1, five dimensions are treated as primary: transfer efficiency, meaning how much data the pipeline can move per second under comparable conditions; latency and pacing stability, meaning not only how fast messages arrive but how regularly they arrive under sustained flow [7]; overload behaviour, meaning whether pressure is absorbed through queue growth, throttling, or admission failure; recovery and correctness, meaning how reconnects, retries, duplicates, and ordering behave when a running pipeline is interrupted; and resource and operational cost, meaning CPU, memory, network traffic, and the practical complexity of running the mechanism in Kubernetes.

This dimensional view is important because it prevents the chapter from collapsing into a single throughput comparison. A mechanism may lead on bulk transport while still performing poorly under restart or overload, and such trade-offs are exactly what DYNAMOS must reason about.

5) Scenario set and scenario-to-property mapping: The full Phase 1 design distinguishes three workload families: *bulk transfer*, *continuous steady-rate flow*, and *bursty transfer*. The currently completed gRPC reference study covers the first two directly and supplements them with two targeted stress probes

that operationalize the properties most relevant to DYNAMOS: slow-consumer backpressure and forced-restart recovery. A bursty profile remains part of the planned cross-technology matrix, but it has not yet been executed in the current gRPC-only reference stage.

6) Experimental factors: The independent variables cover the stress dimensions most frequently discussed in streaming-system and IPC benchmarking literature [16], [22], [29]: dataset size, to expose when buffering or backlog effects become significant; chunk or message size, to test the trade-off between per-message overhead and coarse-grained transfer latency; number of concurrent flows, to assess how each mechanism handles simultaneous transfers; pipeline depth, fixed here at a three-service chain so that raw transport overhead and multi-hop effects can still be distinguished; consumer speed mismatch, to expose backlog behaviour and flow-control effectiveness; resource budget, through explicit CPU and memory requests and limits for Kubernetes pods; durability mode and replication factor, where the mechanism supports such trade-offs; and fault scenario, at minimum service restart during an ongoing transfer.

Not every factor needs to be crossed exhaustively. A pragmatic design is to define a core matrix over mechanism, workload profile, resource budget, and concurrency, and then use targeted follow-up experiments for failure and durability scenarios. This keeps the study manageable while still covering the trade-offs the literature treats as most important [22], [23].

7) Result presentation and appendix boundary: The body of the thesis should present only the compact experiment matrix and a smaller set of focused figures that are necessary for the argument. A master table can summarize each run by mechanism, workload profile, dataset size, message size, concurrency level, resource budget, and durability mode, together with the most important metrics. The main text should then highlight the most informative sweeps, such as throughput versus concurrency, latency percentiles versus message size, and recovery behaviour by failure scenario. Full preset definitions, per-run outputs, and secondary plots should be moved to an appendix so that the chapter remains analytically clear rather than turning into an undifferentiated lab log.

F. Phase 1 Preliminary Results: gRPC Reference Baseline

1) Why gRPC is evaluated first: An initial implementation of the common benchmark has already been completed for gRPC. At this stage, the comparison is intentionally narrow: it contrasts *client-streaming gRPC* with *unary gRPC* in the same three-stage pipeline (*Producer* $\rightarrow$ *Transformer* $\rightarrow$ *Sink*), using identical business logic and payload schemas. This is useful because unary gRPC approximates the current direct request-response style used in DYNAMOS, whereas client-streaming represents the most direct streaming extension of that model. The broader comparison with Kafka, RabbitMQ, and NATS is still pending, so the results below should be read as a *reference point within the gRPC family*, not yet as the final answer to the thesis question.

Two clean workload families were executed for this reference matrix. The first uses synthetic payloads with message sizes from 256 bytes to 16 KiB and concurrency levels from 1 to 16. The second is a smaller realism check that replays rows from the copied DYNAMOS CSV datasets by batching either 25 or 100 rows into one message. For each run, the sink records throughput, end-to-end latency percentiles, and correctness counters. In addition to these clean matrices, three focused stress scenarios were executed to probe the properties identified earlier: a continuous steady-rate jitter scenario, a slow-consumer backpressure scenario, and a recovery scenario with a forced mid-run transformer restart.

For readers less familiar with streaming benchmarks, two metrics are especially important. *Throughput* measures how much data the system can move per second, so higher values are better for large transfers. The p_{95} *latency* measures a near-worst-case delivery delay: 95% of messages are delivered faster than this value, so lower values indicate more consistently responsive communication.

2) Clean baseline matrix: Several clear observations already follow from Table II and Fig. 2. First, client-streaming gRPC is much better at moving large amounts of data through the pipeline. In the synthetic workload, client-streaming achieved roughly 13.5 times the average throughput of unary gRPC. In the CSV replay workload, the same pattern remained visible, with client-streaming achieving roughly 12.5 times the average throughput of unary. This strongly supports the idea that repeatedly opening many individual request-response exchanges adds substantial overhead when the main goal is bulk transfer.

Second, unary gRPC produced much lower p_{95} latency than client-streaming in both workload families. This means unary calls are more responsive on a per-message basis, even though they are much less efficient for sustained high-volume transfer. In practical terms, unary gRPC appears more suitable for small control-style interactions or isolated requests, while client-streaming is more suitable for continuously sending many records to the next service in the pipeline.

Third, the clean baseline matrices did not reveal correctness problems in the implemented gRPC benchmark: no duplicate deliveries or ordering violations were observed in those completed bulk-transfer runs. This is important because raw throughput alone would not be useful if it came at the cost of lost ordering or inconsistent delivery.

The clean matrices alone, however, are not enough to judge whether a transport is suitable for long-lived, policy-constrained data exchange. DYNAMOS-style workflows also care about pacing regularity, behaviour under consumer mismatch, and failure recovery. Three focused stress scenarios were therefore added after the baseline.

3) Focused stress scenarios:

a) Continuous steady-rate scenario.: The first focused experiment was a continuous steady-rate scenario. It used 1 KiB synthetic messages, a fixed target rate of 1,000 messages per second, and concurrency levels of 4 and 8, without artificial sink delay or injected failures. The goal was to measure not

Table I
PHASE 1 SCENARIO-TO-PROPERTY MAPPING.

Scenario	Primary property	Why it matters for DYNAMOS
Bulk transfer baseline	Protocol overhead, sustained throughput, multi-hop transfer efficiency	Large dataset exchange should remain efficient when data is moved incrementally through a composed service chain.
Continuous steady-rate flow	Latency stability, inter-arrival regularity, pacing precision, CPU cost	Long-lived flows matter when data is produced incrementally and consumers depend on relatively stable delivery rhythm.
Slow-consumer backpressure	Queue build-up, overload propagation, throttling versus delay accumulation	DYNAMOS pipelines may contain stages with unequal service rates, so overload handling must be explicit and interpretable.
Forced-restart recovery	Reconnect behaviour, duplicates, ordering preservation, recovery time	Ephemeral microservices and restarts are normal in Kubernetes-style deployments, so transport behaviour under interruption is a first-class concern.

Placeholder for a manual conceptual figure. Suggested content: a simple Phase 1 evaluation map with the common pipeline *Producer* → *Transformer* → *Sink* at the centre, surrounded by four labeled probes: *bulk transfer* (transfer efficiency), *continuous steady-rate* (pacing stability), *slow-consumer backpressure* (overload response), and *forced-restart recovery* (reconnect and duplicate behaviour). The figure should visually communicate that the scenario suite is a literature-guided operationalization of evaluation dimensions rather than an arbitrary collection of tests.

Figure 1. Optional conceptual figure for the Phase 1 scenario design.

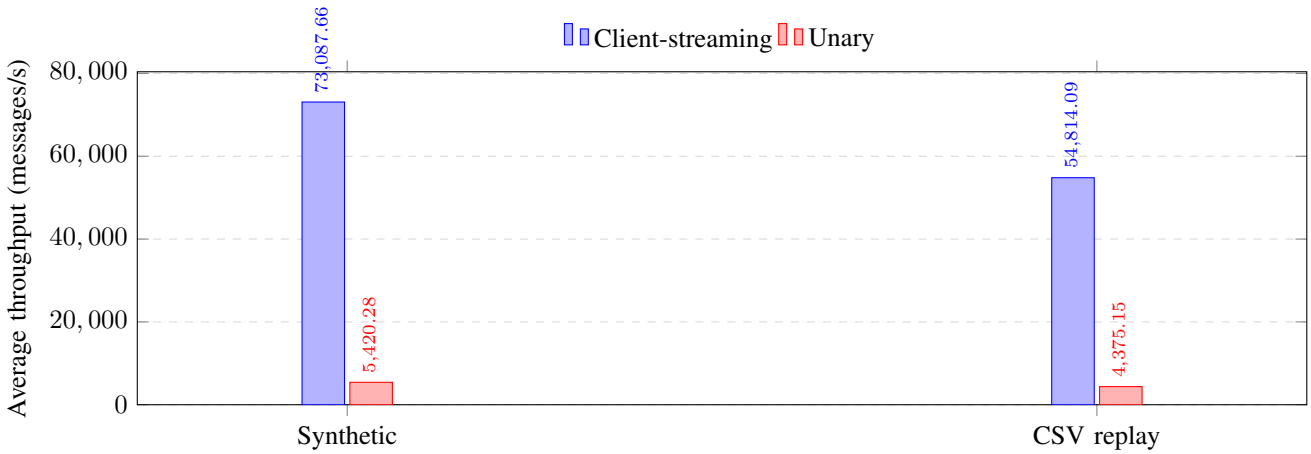

Figure 2. Average throughput for the preliminary gRPC baseline, split by workload family and transport mode. Client-streaming sustains substantially higher throughput than unary gRPC in both workload families.

Table II
PRELIMINARY GRPC BASELINE RESULTS AVERAGED OVER THE COMPLETED RUN MATRICES.

Workload	Transport mode	Avg. throughput (msg/s)	Avg. p_{95} latency (ms)
Synthetic	Client-streaming	73,087.66	42.82
Synthetic	Unary	5,420.28	1.44
CSV replay	Client-streaming	54,814.09	19.24
CSV replay	Unary	4,375.15	1.06

just throughput and latency, but also inter-arrival regularity under a long-running steady stream.

The continuous results show a clear split between delivery regularity and per-message responsiveness. Client-streaming maintained the tighter inter-arrival pattern in both concurrency settings: its jitter stayed around 0.66 ms and its p_{95} inter-arrival delay remained near 1.6 ms. Unary delivered much lower end-to-end latency, but its inter-arrival spread was wider and, at concurrency 8, it exceeded the requested send rate by reaching roughly 1,050 messages per second. In other words, the current client-streaming implementation paces the flow more steadily, whereas unary remains the faster low-latency option. The CPU data also points in the same direction: unary required substantially more transformer CPU than client-streaming in this continuous scenario, indicating that the lower latency was not free.

b) Slow-consumer backpressure scenario.: The second focused experiment was a backpressure scenario. It used the same 1 KiB payload size with concurrency 8, but the sink was slowed down by adding a 2 ms processing delay per message while the producer still targeted 4,000 messages per second. This scenario was designed to expose whether the transport absorbs overload through explicit throttling or through queue

Table III
FOCUSED GRPC CONTINUOUS-STREAM JITTER SCENARIO.

Transport mode	Concurrency	Throughput (msg/s)	Avg. p_{95} (ms)	p_{95} inter-arrival (ms)	Jitter (ms)
Client-streaming	4	918.20	4.68	1.61	0.66
Client-streaming	8	918.70	6.57	1.62	0.66
Unary	4	918.42	1.13	2.17	1.02
Unary	8	1,050.39	1.03	2.16	0.88

growth and accumulated delay.

This result is important because it shows that higher throughput is not automatically the better outcome. Under backpressure, client-streaming kept much more of the nominal input rate, but it did so by allowing the pipeline to accumulate a very large queue. The median end-to-end latency increased to more than 1.4 seconds and the p_{95} latency reached roughly 1.76 seconds. Unary behaved almost oppositely: it gave up throughput, but kept latency near the sub-millisecond range. The interpretation is that client-streaming preserves bulk movement under pressure, whereas unary currently protects responsiveness by throttling harder. For DYNAMOS this is a meaningful design trade-off, because some archetypes may prefer sustained throughput while others may require bounded latency or tighter admission control.

c) Forced-restart recovery scenario.: The third focused experiment was a recovery scenario. This run used 4 KiB messages, concurrency 8, a continuous target rate of 2,000 messages per second, and a forced *Transformer* restart after 3 seconds. The producer was allowed to retry up to 40 times with a 250 ms backoff. The purpose of this scenario was not to replace the main matrix, but to observe how the two gRPC transport styles behave when the pipeline is interrupted during an active stream.

The recovery results change the interpretation in an important way. Unary gRPC sustained the intended rate more effectively after the restart and kept a much lower p_{95} latency than client-streaming. It also completed the run without duplicates, because each request could simply be retried individually once the transformer became reachable again. In that sense, unary currently behaves more cleanly under restart in this benchmark implementation.

However, unary also showed a much larger worst-case stall: its maximum latency exceeded 1.27 seconds, whereas client-streaming remained below 33 ms. Client-streaming also showed slightly lower inter-arrival jitter and a shorter average recovery interval. The trade-off is that the current client-streaming recovery strategy replays a worker's sequence set after reconnect, which preserves completeness but introduces at-least-once behaviour under failure. This explains the 7,163 duplicate messages recorded in the streaming recovery run.

The resource metrics reinforce this distinction. In the recovery preset, unary consumed substantially more CPU across all three roles, especially in the transformer stage, where average sampled CPU usage exceeded 111% compared with roughly 44% for client-streaming. This suggests that unary recovered the flow more aggressively, but at a higher processing cost. Client-streaming, by contrast, reduced CPU pressure at the cost of duplicate replay and worse tail latency.

4) Phase 1 interpretation: The main conclusion at this stage is therefore more precise than a simple "streaming beats unary" statement. In the clean bulk-transfer baseline, client-streaming remains the stronger choice for high-volume inter-service data movement, while unary remains the lower-latency option for small request-response exchanges. In the continuous scenario, client-streaming also provides the steadier delivery rhythm, whereas unary provides lower latency at higher CPU cost and with less precise pacing. In the backpressure scenario, client-streaming preserves throughput by absorbing overload as queueing delay, while unary sacrifices throughput to protect latency. Under restart stress, unary is currently the operationally cleaner implementation in this prototype, whereas client-streaming still needs a more selective resume or deduplication strategy before it can claim equally strong recovery behaviour.

This is a more useful intermediate result for the thesis because it does not reduce the comparison to one metric. Instead, it identifies distinct transport strengths in bulk transfer, pacing regularity, overload response, and failure recovery before Kafka, RabbitMQ, and NATS are added. That is exactly why Phase 1 should be read as a literature-guided scenario evaluation: the properties being tested are grounded in prior work, but their concrete operationalization has been tailored here to DYNAMOS-relevant communication behaviour.

Once the broker-based mechanisms have been benchmarked under the same workload definitions, the thesis will be able to assess whether their additional features, such as replay, decoupling, and backlog tolerance, justify their extra operational complexity relative to the gRPC reference baseline.

IV. PHASE 2: STREAMING FEASIBILITY OF DYNAMOS DATA-SHARING ARCHETYPES

Phase 2 will examine how the selected streaming communication model aligns with the data-sharing archetypes implemented by DYNAMOS. The central question is not only whether a streaming mechanism performs well in isolation, but whether it can be reconciled with the trust boundaries, orchestration rules, and policy assumptions that define DYNAMOS as middleware. This phase will build on the literature and early design analysis already outlined in the proposal and will connect streaming semantics to the system's archetype-based execution model.

Table IV
FOCUSED GRPC BACKPRESSURE SCENARIO WITH AN INTENTIONALLY SLOW SINK.

Transport mode	Throughput (msg/s)	Median latency (ms)	Avg. p_{95} (ms)	p_{95} inter-arrival (ms)	Jitter (ms)
Client-streaming	3,395.94	1,427.58	1,760.95	2.07	0.63
Unary	2,386.17	0.44	0.83	3.18	1.06

Table V
FOCUSED GRPC RECOVERY SCENARIO WITH A FORCED TRANSFORMER RESTART.

Transport mode	Throughput (msg/s)	Avg. p_{95} (ms)	Max latency (ms)	Duplicates	Avg. recovery (ms)
Client-streaming	1,376.11	12.33	32.93	7,163	1,003.14
Unary	2,097.03	2.74	1,274.57	0	1,273.31

V. PHASE 3: INTEGRATION DESIGN FOR DYNAMOS

Phase 3 will translate the findings from the comparative study into an architectural integration design for DYNAMOS. This phase will specify how streaming channels are created, how sidecar responsibilities are divided, how policy enforcement boundaries are preserved, and how long-lived communication interacts with dynamic service composition and isolation guarantees. The output of this phase will be a documented integration design that can guide implementation decisions in a defensible way.

VI. PHASE 4: PROTOTYPE IMPLEMENTATION AND SYSTEM-LEVEL EVALUATION

Phase 4 will implement the selected streaming approach in DYNAMOS and evaluate it against the current unary request–response baseline. The focus will be on both feasibility and system behaviour: how much code and architectural change are required, whether policy-driven orchestration remains intact, and how performance, robustness, and scalability change once streaming is introduced into the middleware itself. The experimental logic prepared in Phase 1 will be reused here, but applied in the full DYNAMOS context rather than in an isolated benchmark pipeline.

VII. PHASE 5: DISCUSSION AND FINAL RECOMMENDATION

Phase 5 will synthesize the evidence from the previous phases into a final discussion of trade-offs and a recommendation for DYNAMOS. This discussion will interpret not only performance outcomes, but also integration complexity, operational implications, compliance considerations, and external validity. The goal is to move from isolated benchmark observations and design proposals toward a justified answer to the thesis-level question of how streaming communication should be integrated into DYNAMOS.

VIII. CONCLUSION

The thesis is structured to move from broad design-space analysis to increasingly system-specific conclusions. At the current stage, the document already contains the core introduction, related work, and the first substantive results from Phase 1. The remaining phases will extend this foundation toward architectural integration, prototype validation, and a final recommendation grounded in both research literature and empirical evidence.

APPENDIX

This appendix section is reserved for the detailed benchmark materials that would otherwise make the Phase 1 chapter too dense. The intended contents are: complete preset definitions, the full run matrix across mechanisms and workloads, per-run summary tables, additional plots not used in the main argument, and configuration details needed for reproducibility. At the current stage of the thesis, these materials exist only for the gRPC reference runs and will be consolidated here once the broker-based comparisons have been executed under the same design.

REFERENCES

[1] J. Stutterheim, "Dynamos: Dynamically adaptive microservice-based os: A middleware for data exchange systems," Master's thesis, University of Amsterdam, 2023.

[2] J. Stutterheim, A. Mifsud, and A. Oprescu, "Dynamos: Dynamic microservice composition for data-exchange systems, lessons learned," in *Proceedings of the IEEE International Conference on Software Architecture (ICSA)*, 2023.

[3] C. Starck and J. Ghofrani, "Improving load balancing of long-lived streaming rpcs for grpc-enabled inter-service communication," in *Proceedings of the 15th Central European Workshop on Services and their Composition (ZEUS 2023)*, ser. CEUR Workshop Proceedings, S. Böhm and D. Lübke, Eds., vol. 3386, Hannover, Germany, 2023, pp. 45–51. [Online]. Available: https://ceur-ws.org/Vol-3386/

[4] A. Sodankoor, A. Shenoy, B. M. Moger, M. Gowda, P. K. Auradkar, and S. Kalambur, "Benchmarking grpc protocol on various virtualization technologies," in *ICT4SD 2025*, ser. Lecture Notes in Networks and Systems. Springer Nature Switzerland AG, 2026, vol. 1654, pp. 316–329.

[5] A. G. Ibrahim, R. P. Lopes, J. Rufino, and P. Leitão, "On the impact of message brokers implementations in the choreography of microservices," in *Open Lecture Series 2A (OL2A 2025)*, ser. Communications in Computer and Information Science (CCIS). Springer Nature Switzerland AG, 2026.

[6] J. Nuikka, "Comparison of cloud native messaging technologies," Master's thesis, Tampere University, Faculty of Information Technology and Communication Sciences, May 2021. [Online]. Available: https://urn.fi/URN:NBN:fi:tuni-202104223314

[7] G. Coviello, K. Rao, C. G. D. Vita, G. Mellone, and S. Chakradhar, "Dataxc: Flexible and efficient communication in microservices-based stream analytics pipelines," in *2022 IEEE Intl Conf on Dependable, Autonomic and Secure Computing (DASC) / PiCom / CBDCom / CyberSciTech*, 2022.
[8] J. Figueira and C. Coutinho, "Developing self-adaptive microservices," *Procedia Computer Science*, vol. 232, pp. 264–273, 2024.
[9] S. Neumaier, G. Havur, and T. Pellegrini, "Towards an architecture for policy-aware decentral dataset exchange," in *Proceedings of the Fifteenth International Conference on Advances in Semantic Processing (SEMAPRO 2021)*. Barcelona, Spain: IARIA, Oct. 2021. [Online]. Available: https://www.researchgate.net/publication/358816711_Towards_an_Architecture_for_Policy-Aware_Decentral_Dataset_Exchange
[10] G. Thiyagarajan, V. Bist, and P. Nayak, "Strengthening grpc security in microservices: A proxy-based approach for mtls, jwt, and rbac enforcement," *International Journal of Computer Applications*, vol. 187, no. 28, Aug. 2025.
[11] J. A. Rasheedh and S. Saradha, "Reactive microservices architecture using a framework of fault tolerance mechanisms," in *Proceedings of the Second International Conference on Electronics and Sustainable Communication Systems (ICESC)*. IEEE, 2021.
[12] B. Guntupalli and S. Vamshi Ch, "Designing microservices that handle high-volume data loads," *International Journal of AI, Big Data, Computational and Management Studies*, vol. 4, no. 4, pp. 76–87, 2023.
[13] Y. Gan and C. Delimitrou, "The architectural implications of cloud microservices," *IEEE Computer Architecture Letters*, vol. 17, no. 2, pp. 155–158, 2018.
[14] Ritu, S. Arora, A. Bhardwaj, A. Kukkar, and S. Kaur, "A comparative analysis of communication efficiency: Rest vs. grpc in microservice-based ecosystems," in *Proceedings of the 2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)*. IEEE, 2024.
[15] V. Lähtevänoja, "Communication methods and protocols between microservices on a public cloud platform," Master's thesis, Aalto University, School of Science, Espoo, Finland, May 2021. [Online]. Available: https://urn.fi/URN:NBN:fi:aalto-202106207501
[16] S. Ris, J. Araujo, and D. Beserra, "A systemic mapping of methods and tools for performance analysis of data streaming with containerized microservices architecture," in *2023 18th Iberian Conference on Information Systems and Technologies (CISTI)*, 2023, pp. 1–6.
[17] gRPC Authors. (2024) Core concepts, architecture and lifecycle. Last modified 2024-11-12. [Online]. Available: https://grpc.io/docs/what-is-grpc/core-concepts/
[18] M. Roohitavaf, K. Ren, G. Zhang, and S. Ben-Romdhane, "Logplayer: Fault-tolerant exactly-once delivery using grpc asynchronous streaming," 2019, arXiv preprint.
[19] Apache Software Foundation. (2026) Apache kafka. [Online]. Available: https://kafka.apache.org/
[20] P. Dobbelaere and K. Sheykh Esmaili, "Kafka versus rabbitmq," 2017, arXiv preprint.
[21] M. Mohammad, "Analysis of design patterns and benchmark practices in apache kafka event-streaming systems," 2025, arXiv preprint.
[22] B. Çiftçi and B. Çiloğlugil, "A review of comparative studies on performance evaluation of communication mechanisms for microservices," in *Computational Science and Its Applications – ICCSA 2025*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland AG, 2025, vol. 15650, pp. 98–113.
[23] T. Sharvari and K. Sowmya Nag, "A study on modern messaging systems: Kafka, rabbitmq and nats streaming," 2019, arXiv preprint.
[24] RabbitMQ Team. (2026) Streams and superstreams (partitioned streams). [Online]. Available: https://www.rabbitmq.com/docs/streams
[25] The NATS Authors. (2026) Overview. [Online]. Available: https://docs.nats.io/nats-concepts/overview
[26] ——. (2026) Request-reply. [Online]. Available: https://docs.nats.io/nats-concepts/core-nats/reqreply
[27] ——. (2026) Jetstream. [Online]. Available: https://docs.nats.io/nats-concepts/jetstream
[28] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proceedings of the IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1519–1530.
[29] S. Henning and W. Hasselbring, "How to measure scalability of distributed stream processing engines?" in *Companion of the ACM/SPEC International Conference on Performance Engineering (ICPE '21)*. ACM, 2021, pp. 85–88.